



PROJETO 1 – FAMILIARIZAÇÃO COM MÁQUINA VIRTUAL E SINCRONIZAÇÃO DE PROCESSOS

TAREFA 2: PROCESSOS (RECAPITULANDO CONCEITOS DE PROCESSOS MEDIANTE ATIVIDADES PRÁTICAS) (de 09/12/2024 a 12/12/2024)

A seguir serão passadas uma série de atividades dirigidas cujo objetivo é recapitular conceitos fundamentais de sistemas operacionais, vistos na disciplina de SO1. **Para cada atividade, apresente o código fonte comentado e entregue este para o docente via SIGAA para análise.**

2.1. Sincronização de processos

Em SO1, muito foi discutido acerca de programação concorrente e cooperativismo entre processos. Foram apresentadas soluções para implementar a sincronização de processos, que permitem acesso a dados compartilhados de forma controlada, evitando inconsistências decorrentes da operação do escalonador de processos. Dessa forma, foram apresentados mecanismos que garantem a execução ordenada de processos cooperativos e por conseguinte, asseguram a manutenção de consistência dos dados (maiores informações nas unidades 2.6 e 2.7 de SO1).

2.1.1 Solução de Peterson

A solução de Peterson é uma solução algorítmica que permite realizar o controle de acesso a seção crítica, utilizando para isso espera ocupada e variável de guarda (Código 1). O Código 2 apresenta duas *threads* p_i e p_j , as quais operam sobre uma variável compartilhada. Pede-se a implementação da solução de Peterson para o controle de acesso a seção crítica, seguindo os moldes apresentados no Código 3. **Responda: 1) caracterize a seção crítica, levando em conta o Código 2 e a proposta de solução apresentada no Código 3; 2) como a solução de Peterson protege a seção crítica identificada?; 3) faça o tratamento de erro de modo a garantir que a *thread* foi criada adequadamente; 4) Como você faria a criação de p_i e p_j utilizando apenas uma seção de código genérica p ?; 5) Entregue o(s) código(s) resultantes dessa prática juntamente com as respostas solicitadas via SIGAA na tarefa apropriada.**

```

do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
        Seção crítica
    flag[i] = false;
        Restante do código
} while (true);

```

Código 1: Base para a implementação da solução de Peterson.

```

...

#define TRUE 1
#define FALSE 0

pthread_t tid[2];
int counter;

void * pi (void *arg){
    int i;

    printf("\nPi: Minha vez!!!\n");
    counter++;
    printf("\n\nPi: counter = %d\n\n", counter);
    return NULL;
}
void * pj (void *arg){
    int i;

    printf("\nPj: Minha vez!!!\n");
    counter++;
    printf("\n\nPj: counter = %d\n\n", counter);
    return NULL;
}
int main (void)
{
    pthread_create (&(tid[0]), NULL, &pi, NULL);
    pthread_create (&(tid[1]), NULL, &pj, NULL);

    pthread_join (tid[0], NULL);
    pthread_join (tid[1], NULL);

    return 0;
}

```

Código 2: Problema proposto.

```

...

#define TRUE 1
#define FALSE 0

...

```

```

void enterSC (int i)
{
    ...

    return;
}

void leaveSC (int i)
{
    ...

    return;
}

void * pi (void *arg){
    enterSC(0); // Entra na região crítica
    printf("\nPj: A região crítica é minha!!!\n");
    counter++;
    printf("\nPj: counter = %d\n", counter);
    leaveSC(0);
    return NULL;
}

void * pj (void *arg){
    enterSC(1); // Entra na região crítica
    printf("\nPj: A região crítica é minha!!!\n");
    counter++;
    printf("\nPj: counter = %d\n", counter);
    leaveSC(1);
    return NULL;
}

int
main (void)
{
    ...

    return 0;
}

```

Código 3: Formato da solução proposta.

2.1.2 Solução de hardware usando locks do tipo test and set

Muitos sistemas oferecem suporte de hardware para implementação de código de acesso a seção crítica (SC). Para este exercício, pede-se que o aluno utilize o a biblioteca *pthread* e a instrução `atomic_flag_test_and_set` para implementar o controle de acesso a seção crítica para as *threads* conforme criadas no Código 3. Esta solução é baseada na ideia de variável de intertravamento (*locking*) e um teste e um ajuste de valor para uma palavra de memória. **Pesquise o funcionamento desta solução e implemente o código de controle de acesso para a seção crítica**

apropriado. Demonstre utilizando como base o problema proposto na Seção 2.1.1.

2.1.3 Solução de hardware usando locks do tipo mutex

As soluções apresentadas anteriormente são um pouco mais complicadas e geralmente inacessíveis a programadores de aplicação. Projetistas de SO disponibilizam ferramentas de software para resolver o problema da seção crítica, como é o caso dos *locks mutex*. No contexto de *threads*, este é um objeto de propriedade desta, o qual permite que apenas uma única thread acesse o recurso ou SC. Utilizando a biblioteca *pthread* e a instrução *pthread_mutex_lock* / *pthread_mutex_unlock*, realize o controle de acesso a SC conforme ilustrado no Código 3.

Pesquise o funcionamento desta solução e implemente o código de controle de acesso para a seção crítica apropriado. Demonstre utilizando como base o problema proposto na Seção 2.1.1.

2.1.4 Solução de hardware usando semáforo

Semáforo é uma ferramenta de sincronização que oferece formas mais sofisticadas (do que *mutex locks*) para que um processo sincronize suas atividades. No contexto dessa atividade, iremos utilizar apenas semáforos binários. Utilizando a biblioteca *semaphore.h* e a instrução *sem_wait/sem_post*, realize o controle de acesso a SC conforme ilustrado no Código 3. Pesquise o funcionamento desta solução e implemente o código de controle de acesso para a seção crítica apropriado. Demonstre utilizando como base o problema proposto na Seção 2.1.1.